INFORME DE RESOLUCIÓN DE CASO: GESTIÓN DE BUFFET UNIVERSITARIO

Nombre: Alan Wetzel

Dni: 46.459.062

Com A

Asignatura: Programación I

Carrera: Ingeniería en Sistemas de Información

Institución: Universidad de la Cuenca del Plata

1. Descripción del Problema y Justificación de Estructuras de Datos

Descripción del Problema

El sistema desarrollado automatiza la gestión semanal de un buffet universitario. Permite registrar un catálogo de productos (código, nombre y precio) controlando que no existan códigos duplicados, registrar las cantidades vendidas de cada producto a lo largo de los cinco días hábiles de la semana mediante una matriz de doble entrada, y procesar informes automáticos sobre el producto más vendido, el estado de inventario/ventas y la recaudación diaria y semanal. Asimismo, cuenta con persistencia de datos mediante archivos de texto (productos.txt y ventas.txt) para evitar la pérdida de información al cerrar el programa.

Justificación de las Estructuras de Datos Seleccionadas

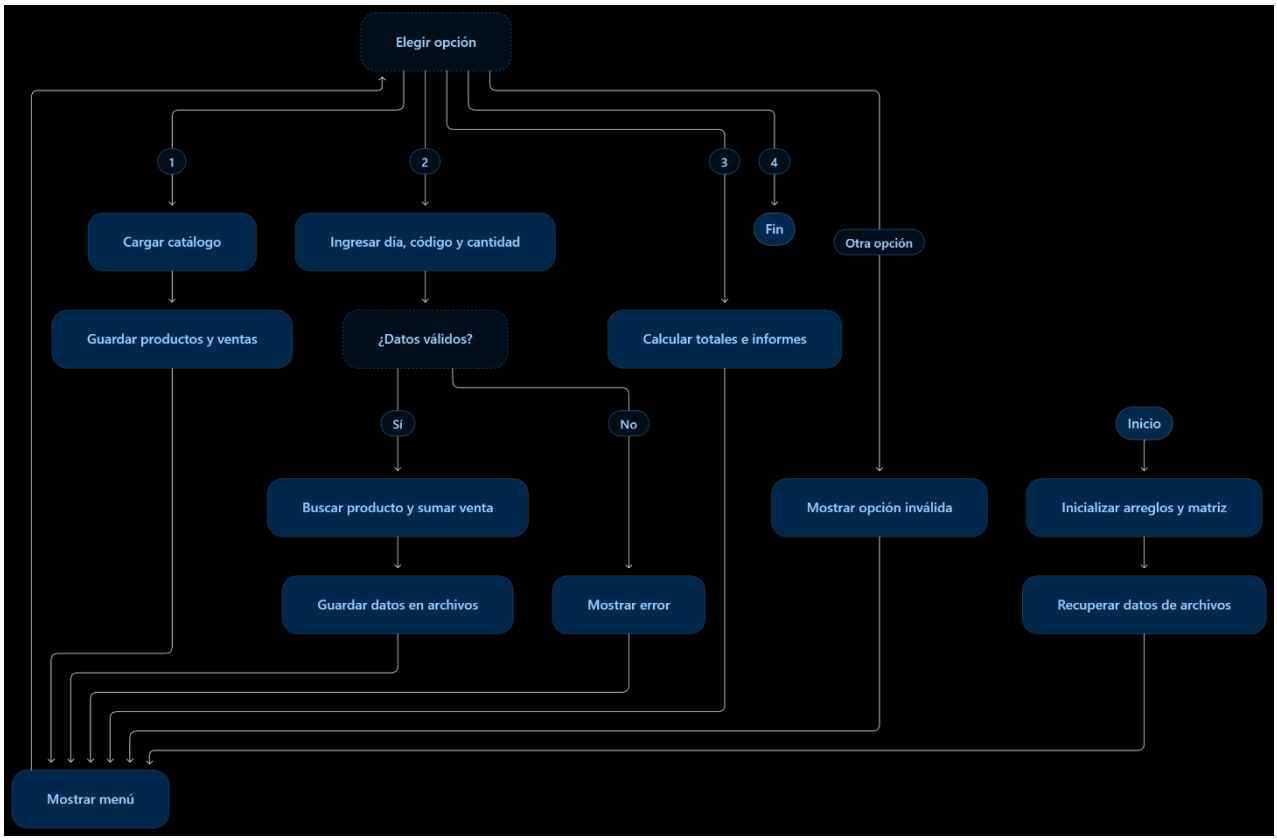
- **Arrays Paralelos (codigos, nombres, precios):** Se implementaron vectores unidimensionales de tamaño fijo (CANT_PRODUCTOS = 6) donde la posición i de cada arreglo corresponde a las características del mismo producto. Se eligieron frente a una única estructura compleja por su simplicidad de recorrido secuencial y búsqueda indexada en esta etapa de la cursada.
- **Matriz Bidimensional (ventas[d][p]):** Se estructuró una lista de listas de dimensiones CANT_DIAS (5) por CANT_PRODUCTOS (6). Su elección frente a múltiples arrays unidimensionales se debe a que modela de forma natural una tabla relacional de doble entrada, donde las filas representan los días de la semana y las columnas representan los productos, facilitando la acumulación y el cálculo de subtotales por día o por producto de manera eficiente mediante bucles anidados.
- **Archivos de Texto (productos.txt y ventas.txt):** Se prefirieron los archivos planos delimitados por punto y coma (;) frente al almacenamiento volátil en

memoria para garantizar la persistencia de los datos entre distintas ejecuciones del programa.

2. Representación Lógica (Pseudocódigo / Diagrama de Lógica)

La lógica principal del sistema opera bajo un menú de opciones iterativo controlado por un bucle while:

1. **Carga de Datos Iniciales:** Al iniciar, el sistema verifica si existen los archivos productos.txt y ventas.txt en el directorio. Si existen, carga los datos a memoria mediante la función cargar_datos().
2. **Menú Principal:**
    - **Opción 1 (Cargar Catálogo):** Solicita códigos únicos (validados para que no sean 0 ni estén repetidos), nombres (reemplazando caracteres conflictivos) y precios, guardándolos automáticamente en disco.
    - **Opción 2 (Registrar Venta):** Permite ingresar el día (1 a 5), el código del producto y la cantidad vendida, acumulando los valores en la matriz de ventas y actualizando el archivo.
    - **Opción 3 (Ver Informes):** Calcula y muestra en consola las unidades vendidas por producto, identifica el producto más popular y el que no registró ventas, calcula la recaudación por día y determina el día de mayor ingresos junto al total semanal.
    - **Opción 4 (Salir):** Finaliza la ejecución del programa.




```
1. Cargar catálogo
2. Registrar venta
3. Ver informes
4. Salir
Opción: C:\Users\djala\AppData\Local\Programs\Python\Python314\pyth
Error, ingrese un número entero: 1
Código del producto 1: 123
Nombre: xxxx
Precio: 3000
Código del producto 2: 321
Nombre: zzzz
Precio: 2000
Código del producto 3: 456
Nombre: eeee
Precio: 5000
Código del producto 4:
Error, ingrese un número entero: 2
Nombre: xxxx
Precio: 3000
```

```
1. Cargar catálogo
2. Registrar venta
3. Ver informes
4. Salir
Opción: 3
xxxx : 0 unidades
   ^ no se vendió
zzzz : 0 unidades
   ^ no se vendió
eeee : 0 unidades
   ^ no se vendió
xxxx : 0 unidades
   ^ no se vendió
2 : 1 unidades
   ^ el más vendido
4 : 0 unidades
   ^ no se vendió
Lunes : $ 3
Martes : $ 0
Miércoles : $ 0
Jueves : $ 0
Viernes : $ 0
Mayor recaudación: Lunes
Total de la semana: $ 3
```

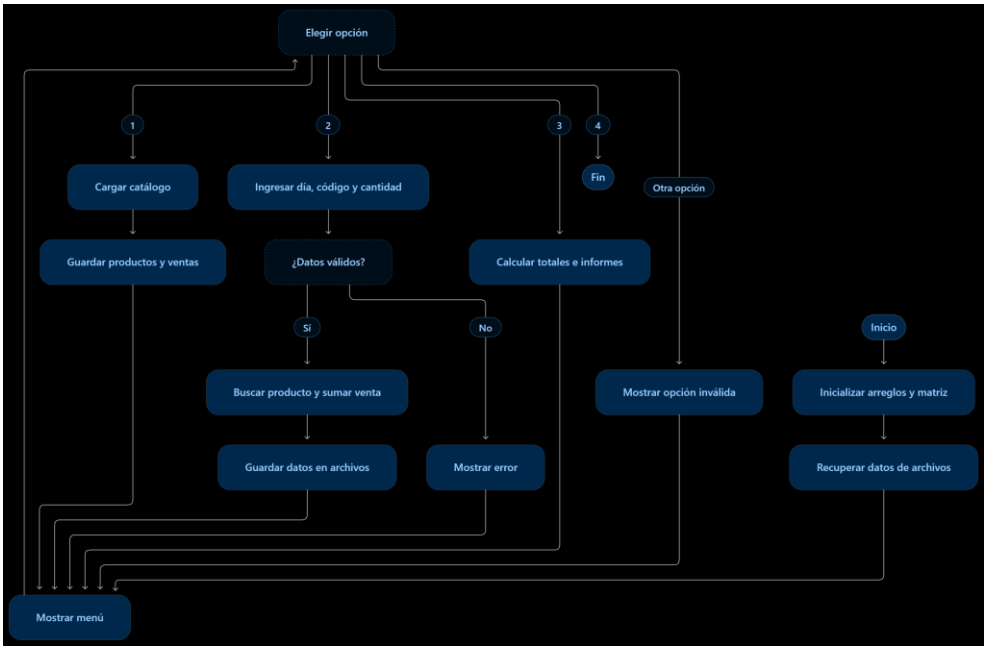
```
1. Cargar catálogo
2. Registrar venta
3. Ver informes
4. Salir
Opción: 2
Día (1 = lunes ... 5 = viernes): 1
Código del producto: zzzz
Error, ingrese un número entero: 1
Cantidad: 1
```

5. Análisis Crítico y Reflexión

Durante el diseño de la solución, se descartó el uso de estructuras basadas en **Diccionarios Anidados u Objetos complejos (POO)** debido a que exceden el alcance metodológico de la unidad actual de Programación I, priorizándose la consolidación lógica mediante arreglos primitivos y matrices.

Como conclusión, el uso de arrays paralelos combinados con matrices bidimensionales demuestra ser sumamente eficiente para volúmenes acotados de datos tabulares; sin embargo, su limitación principal radica en la rigidez del tamaño estático (CANT_PRODUCTOS = 6), lo que exige conocer de antemano la dimensión máxima del dominio de datos. La incorporación de archivos de texto simples resolvió con éxito la persistencia básica, destacando la importancia de validar de forma rigurosa los tipos de datos de entrada para evitar fallas en tiempo de ejecución.

Portafolio Digital – Sistema de gestión del buffet

1. Selección de estructuras de datos
   Para realizar el programa del buffet elegí utilizar arreglos paralelos y una matriz de ventas. Los arreglos paralelos guardan el código, el nombre y el precio de los seis productos, relacionando los datos mediante la misma posición. La matriz registra las cantidades vendidas de cada producto durante los cinco días de la semana. Elegimos estas estructuras porque permiten organizar la información, registrar las ventas y calcular los totales de forma sencilla. Además, utilizamos archivos de texto para conservar los datos cuando se cierra el programa.
2. 

El diagrama representa el funcionamiento general del sistema del buffet. Primero, se inicializan las estructuras de datos y se recupera la información guardada en los archivos. Luego, se muestra un menú que permite cargar el catálogo, registrar una venta, consultar informes o salir del programa. Para registrar una venta, se ingresan el día, el código del producto y la cantidad; se verifica que los datos sean válidos y, si lo son, se actualiza la matriz de ventas y se guardan los cambios en los archivos.
Este diagrama es un borrador del diseño que muestra los pasos principales de la solución antes de la implementación completa en Python.

3. Primer avance de código

Este fragmento representa la carga inicial del catálogo y el control de códigos repetidos, tomando como referencia la función cargar_catalogo() de la implementación en Python.

```python
CANT_PRODUCTOS = 6

codigos = [0] * CANT_PRODUCTOS
nombres = [""] * CANT_PRODUCTOS
precios = [0] * CANT_PRODUCTOS

for i in range(CANT_PRODUCTOS):
    codigo = int(input("Ingrese el código: "))

    while codigo == 0 or codigo in codigos:
        codigo = int(input("Código inválido o repetido. Ingrese otro: "))

    codigos[i] = codigo
    nombres[i] = input("Ingrese el nombre: ")
    precios[i] = int(input("Ingrese el precio: "))

print("Catálogo cargado correctamente")
```

PRUEBAS:

```
1. Cargar catálogo
2. Registrar venta
3. Ver informes
4. Salir
Opción: C:\Users\djala\AppData\Local\Programs\Python\Python314\pyth
Error, ingrese un número entero: 1
Código del producto 1: 123
Nombre: xxxx
Precio: 3000
Código del producto 2: 321
Nombre: zzzz
Precio: 2000
Código del producto 3: 456
Nombre: eeee
Precio: 5000
Código del producto 4:
Error, ingrese un número entero: 2
Nombre: xxxx
Precio: 3000
```

```
1. Cargar catálogo
2. Registrar venta
3. Ver informes
4. Salir
Opción: 2
Día (1 = lunes ... 5 = viernes): 1
Código del producto: zzzz
Error, ingrese un número entero: 1
Cantidad: 1
```

```
1. Cargar catálogo
2. Registrar venta
3. Ver informes
4. Salir
Opción: 3
xxxx : 0 unidades
   ^ no se vendió
zzzz : 0 unidades
   ^ no se vendió
eeee : 0 unidades
   ^ no se vendió
xxxx : 0 unidades
   ^ no se vendió
2 : 1 unidades
   ^ el más vendido
4 : 0 unidades
   ^ no se vendió
Lunes : $ 3
Martes : $ 0
Miércoles : $ 0
Jueves : $ 0
Viernes : $ 0
Mayor recaudación: Lunes
Total de la semana: $ 3
```